Integrating AI Clusters into Virtual Private Cloud

Yinhe Wang*
Computer Network Information Center, Chinese Academy of Sciences
Beijing, China
University of Chinese Academy of Sciences
Beijing, China
Alibaba Cloud
Hangzhou, China
yhwang2025@cnic.cn

Xing Li*
Zhejiang University
Hangzhou, China
Alibaba Cloud
Hangzhou, China
lixing.lix@aliyun-inc.com

Enge Song
Alibaba Cloud
Hangzhou, China
enge.seg@alibaba-inc.com

Zihao Fan
Alibaba Cloud
Hangzhou, China
fanzihao.fzh@alibaba-inc.com

Changgang Zheng
Alibaba Cloud
Hangzhou, China
zhengchanggang.zcg@alibaba-inc.com

Haonan Li
Alibaba Cloud
Hangzhou, China
wenxu.lhn@alibaba-inc.com

Shengyao Gao
Hangzhou Institute for Advanced Study, University of Chinese Academy of Sciences
Hangzhou, China
gaoshengyao25@mails.ucas.ac.cn

Juncheng Xiang
Computer Network Information Center, Chinese Academy of Sciences
Beijing, China
University of Chinese Academy of Sciences
Beijing, China
jcxiang@cnic.cn

Ye Yang
Alibaba Cloud
Hangzhou, China
huaizhou.yy@alibaba-inc.com

Junnan Cai
Alibaba Cloud
Hangzhou, China
caijunnan.cjn@alibaba-inc.com

Chang Liu
Alibaba Cloud
Hangzhou, China
qionglong.lc@alibaba-inc.com

Haoxiang Pan
Alibaba Cloud
Hangzhou, China
panhaoxiang.phx@alibaba-inc.com

Yang Song
Alibaba Cloud
Hangzhou, China
song288954@alibaba-inc.com

Yilong Lv
Alibaba Cloud
Hangzhou, China
lvyilong.lyl@alibaba-inc.com

Qiang Fu
School of Computing Technologies, RMIT University
Melbourne, Australia
qiang.fu@rmit.edu.au

Zhigang Zong
Alibaba Cloud
Hangzhou, China
xuanmin.zzg@alibaba-inc.com

Heng Pan†
Computer Network Information Center, Chinese Academy of Sciences
Beijing, China
panheng@cnic.cn

Shunmin Zhu†
Alibaba Cloud
Hangzhou, China
jianghe.zsm@alibaba-inc.com

Abstract

While commodity NIC-based back-end AI networks offer ultra-high intra-cluster bandwidth for distributed training, their limited programmability and on-chip resources hinder the implementation of advanced VPC features such as fine-grained isolation and stateful security policies. Furthermore, access to resources within the VPC needs to be routed through the front-end DPU, which is shared by the scale-up domain. The mismatch between the front-end DPU's bandwidth and the back-end requirements causes GPU underutilization when intensive VPC communication is required for content recommendation, AIGC, and federated learning workloads. We propose an architecture that decouples complex policy enforcement from high-speed packet forwarding to support VPC semantics on back-end NICs and enable front-end/back-end integration. Evaluations show near-full GPU utilization in our analytical model and

*Co-first authors. †Co-corresponding authors.

APNet '26, Singapore, Singapore

ACM ISBN 979-8-4007-2664-4/26/08
https://doi.org/10.1145/3820441.3820459

71 μs P999 extra latency of the first packet, suggesting that commodity hardware can support both high-throughput AI training and flexible VPC features.

CCS Concepts

• **Networks** → **Cloud computing**; **Data center networks**; **Network design principles**.

Keywords

AI clusters, Virtual Private Cloud, RDMA, cloud networking

ACM Reference Format:

Yinhe Wang, Xing Li, Enge Song, Zihao Fan, Changgang Zheng, Haonan Li, Shengyao Gao, Juncheng Xiang, Ye Yang, Junnan Cai, Chang Liu, Haoxiang Pan, Yang Song, Yilong Lv, Qiang Fu, Zhigang Zong, Heng Pan, and Shunmin Zhu. 2026. Integrating AI Clusters into Virtual Private Cloud. In *The 10th Asia-Pacific Workshop on Networking (APNet '26), August 6–7, 2026, Singapore, Singapore.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3820441.3820459

1 Introduction

Modern AI clusters commonly separate the network into a front-end and a back-end [14, 15, 47]. The front-end network handles control-plane traffic, such as task scheduling and access to virtual private cloud (VPC) services (e.g., storage services), while the back-end network provides high-throughput communication among compute nodes for distributed model training and inference. This separation was originally designed to improve scalability and efficiency.

However, as large-scale AI workloads increasingly demand VPC features such as tenant isolation, access control lists (ACLs) [5, 10, 16, 29], and security groups [1, 7, 18, 30], the separated back-end network faces severe flexibility challenges. In practice, the back-end network often uses commodity network interface cards (NICs) with limited programmability and on-chip resources, which prevents them from storing fine-grained rule tables or performing the local rule lookups required to enforce VPC features. At the same time, bare-metal deployments require provider-managed network functions to stay outside tenant host resources (§2.1), so these VPC semantics must be supplied by the infrastructure rather than by host software. As a result, back-end deployments often resort to Virtual Local Area Network (VLAN)-based isolation, a method dependent on static switch configurations, inflexible to dynamic tenant changes, and unable to enforce granular stateful policies.

Beyond flexibility, the separated architecture also introduces performance bottlenecks under modern AI workloads. With the rapid growth of cloud-based services such as content recommendation [26, 53], autonomous driving [27, 52], AI-generated content (AIGC) [12, 24], and federated learning [17, 25], the underlying model training demands driven by these services have expanded dramatically. Consequently, training datasets have surged to terabytes or even petabytes. In the separated architecture, training nodes need to fetch massive datasets from the VPC through the front-end data processing unit (DPU) (see Figure 1), as the back-end NIC natively lacks support for large rule tables and the flexible table lookup required by the VPC. However, the bandwidth of the front-end DPU is much lower than the aggregate bandwidth of the back-end network formed by multiple NICs. This significant

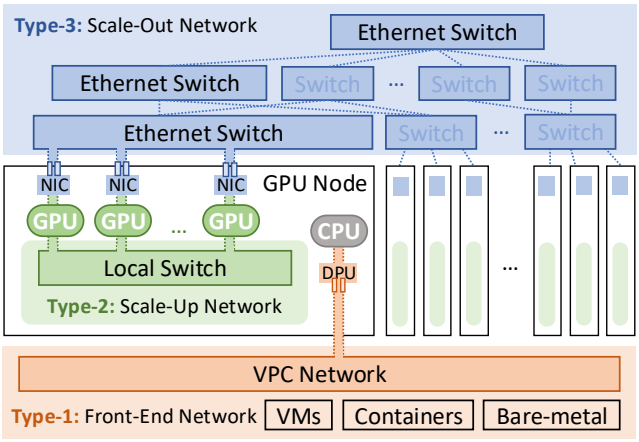

Figure 1: Front-end and back-end (scale-up and scale-out) separation architecture in large-scale AI clusters.

bandwidth disparity turns the front-end DPU into a severe I/O bottleneck, causing powerful GPU clusters to idle while waiting for data, which greatly reduces training efficiency.

These challenges highlight that the existing front-end and back-end separation architecture does not naturally provide both high performance for AI clusters and VPC-facing semantics. We therefore explore how to integrate the front-end and back-end systems at the forwarding architecture level. To this end, we propose a VPC-facing back-end architecture that decouples policy enforcement from high-speed packet forwarding to overcome the limitations of on-chip programmability and resource constraints. This approach avoids requiring every back-end NIC to be replaced by a more powerful but more expensive DPU.

Realizing this architecture introduces practical implementation challenges. Since the offloading mechanism redirects unmatched traffic to a controller, it adds first-packet latency on the slow path. Reliability is also a critical concern, as a failure in the centralized control plane could affect a large number of GPU nodes. Furthermore, ensuring consistency between cached match-action entries and dynamic rule updates is complex, particularly given the traffic volume of AI workloads. The design therefore needs to minimize match-miss upcalls to avoid overloading the controller. This paper presents an initial architecture and preliminary evaluation for addressing these challenges. The simulation results suggest that removing the VPC I/O bottleneck can boost the compute utilization of the AI cluster from 11.7% to nearly 100%. Meanwhile, a hardware upcall experiment measures 71 μs P999 extra latency for first-packet rule installation.

2 Background and Motivation

2.1 AI Cluster Architecture

2.1.1 Front-end and Back-end Separation. In modern large-scale AI training and inference clusters, the front-end and back-end separation architecture decouples the control plane (front-end network) from the data and computation plane (back-end network) [14]. This design enhances scalability, flexibility, and resource efficiency by isolating control-related traffic, such as global scheduling and status monitoring, from high-throughput data communication critical to

Table 1: Characteristics of front-end and back-end networks.

	Front-end	Back-end
Bandwidth	~ hundreds Gbps	~ Tbps
Latency	~ milliseconds	~ microseconds
Functionality	Connect VPCs	Connect AI nodes

distributed model execution. As illustrated in Figure 1, the front-end network connects the user VPC to external systems and orchestrates cluster-level functions such as task scheduling, metadata synchronization, and system monitoring. It typically uses data-center-wide interconnects and links storage [6, 9, 21, 32], VMs [3, 8, 20, 33], and other orchestration components. In contrast, the back-end network provides the high-throughput communication fabric for model execution. It includes the scale-out network for inter-node communication, such as gradient exchange and tensor partitioning, and the scale-up network for intra-node GPU communication (e.g., NVLink [43]).

These networks exhibit distinct characteristics [14, 34], as shown in Table 1. The front-end network prioritizes reliability and coordination efficiency, tolerating millisecond-level latencies and requiring per-node bandwidth on the order of hundreds of Gbps (e.g., Alibaba Cloud's 200 Gbps links [28]). The back-end network emphasizes high availability and performance, demanding microsecond- or even sub-microsecond-scale latency and up to terabits-per-second bandwidth to sustain large-scale parallel training workloads (e.g., NVIDIA's Tbps Ethernet or InfiniBand [47]).

2.1.2 NIC Mode for Scale-Out Network. The scale-out network in large-scale AI clusters is typically configured to use commodity NICs rather than DPUs [34]. This design choice reflects a balance between performance requirements, ecosystem maturity, deployment simplicity, and overall cost efficiency.

Performance Considerations. NICs are purpose-built for high-throughput, low-latency communication [35, 38, 39]. They support high-performance transport protocols such as InfiniBand and RoCEv2, and enable zero-copy transmission with memory-access-like performance via RDMA. Their lightweight design minimizes computation and control logic on the data path, allowing efficient packet forwarding without the overhead of protocol processing or software stack management. In contrast, DPUs integrate general-purpose compute cores, memory controllers, and programmable engines designed for offloading complex network functions such as encryption, storage orchestration, or firewalls [36, 46]. While this flexibility is advantageous in front-end or storage networks, it introduces unnecessary complexity and resource overhead for the minimal and performance-sensitive data movement patterns typical of AI training workloads.

Deployment and Software Ecosystem. NICs have a mature software ecosystem [35, 38, 39] with well-established tools and libraries designed for AI workloads (e.g., NCCL [41], Horovod [22], UCX [49]), requiring minimal configuration for RDMA or RoCEv2. In contrast, DPUs [36, 46] require vendor-specific drivers, firmware maintenance, and dedicated software stacks (e.g., NVIDIA DOCA [42], DPDK [48]), adding significant operational overhead. Current support for DPUs in mainstream AI frameworks remains partial and fragmented, with limited transparency and portability.

Cost and Practicality. Compared with DPUs, NICs are significantly cheaper and already widely deployed in existing high performance computing (HPC) and AI clusters, which enables straightforward hardware reuse. By contrast, DPU deployment typically requires custom firmware, specific orchestration frameworks, and specialized expertise, which increases integration complexity and complicates upgrades and maintenance. Therefore, NIC-based configurations generally provide lower total cost of ownership (TCO) and better compatibility with mainstream AI software stacks.

2.1.3 Tenant Isolation on Bare Metal. Bare-metal servers [2, 19, 31] provide physical machines directly to tenants without virtualization or container abstraction, thereby offering stronger resource exclusivity and more predictable performance for large-scale training of large language models (LLMs). In typical cloud deployments, each server contains multiple high-performance GPUs (e.g., A100 or H100) interconnected via NVLink to form a high-bandwidth training node. Accordingly, users can directly access the host's CPU, GPU, memory, and storage, while provider-managed network virtualization is typically handled by infrastructure components such as the front-end DPU. To ensure tenant isolation in bare-metal settings, the cloud provider sells each server as an indivisible unit, assigning all internal resources to a single tenant.

2.2 Limitations of Existing Solutions

2.2.1 VLAN-Based Isolation. The limited on-chip resources make VLAN-based isolation a natural choice for scale-out networks. It isolates inter-tenant traffic using static identifiers such as VLAN tags or tenant IDs embedded in IPv6 headers. However, it has several key limitations.

Heavy Reliance on Switch Configuration. VLAN-based isolation depends on underlying switches to enforce isolation and policy control, such as VLAN tag assignments and ACL [5, 10] rule matching. This places a heavy functional burden on the switches. In large-scale clusters with multiple layers of switches, configurations must be propagated across many devices, making the process complex and time-consuming. Moreover, any tenant change or workload migration requires synchronized reconfiguration, leading to high operational overhead and poor flexibility.

Lack of Security Group Support. The scale-out network cannot support security groups because it relies solely on physical switches to implement VPC functionalities, such as VLAN-based isolation or basic access control through static ACL rules. While these mechanisms enable coarse-grained and static isolation, they suffer from two fundamental limitations. First, switches operate on a per-packet basis and do not maintain connection state, making it impossible to support stateful security policies, such as allowing return traffic based on ingress rules. Second, switch rules are statically bound to ports or VLANs and cannot dynamically adapt to tenant changes or reflect tenant-specific security policies. As a result, the scale-out network lacks the flexibility, granularity, and stateful control required to enforce security groups.

2.2.2 Front-end DPU I/O Bottleneck. Commodity NICs do not directly connect to VPC services because they lack the programmability and resources needed to maintain tenant-specific rule tables. As a result, data ingress typically goes through the front-end DPU.

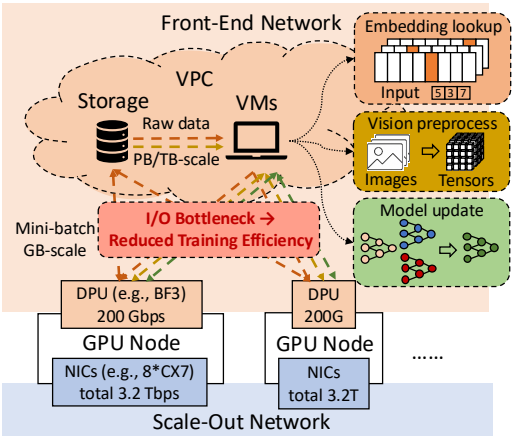

Figure 2: Front-end DPU I/O bottleneck in training pipelines.

This path can become a severe bottleneck in large-scale cloud AI systems. Compared to the scale-out network (e.g., 8×400 Gbps), the front-end network often reaches in-VPC services through a single 200–400 Gbps DPU, creating a significant bandwidth disparity that leaves GPUs idle during data ingestion. Although production data ingestion also depends on storage, preprocessing, memory, and network resources [54], this paper focuses on the network-facing part of the pipeline between cloud-resident resources and high-bandwidth AI back-ends.

Data-Intensive Training Workloads. Various training paradigms impose heavy pressure on the front-end DPU. In *Recommendation Models*, training nodes fetch user-behavior data from storage and concurrently access in-VPC embedding services through the front-end DPU (orange workflow in Figure 2), often saturating its bandwidth. Similarly, *Multi-Modal Training* requires ingesting petabyte-scale raw data preprocessed on in-VPC VMs (dark yellow workflow), while *Federated Learning* demands frequent parameter exchanges for secure aggregation (green workflow). In these scenarios, the limited front-end bandwidth causes data transfer to lag behind computation, significantly reducing overall efficiency.

LLM Inference. The bottleneck is even more pronounced in long-context agentic inference. Recent studies [51] show that with key-value (KV) cache hit rates typically exceeding 95%, the workload shifts from computation to massive data retrieval. In such scenarios, the retrieval of matched KV entries during each prefill step can place substantial pressure on the front-end DPU, particularly as context length and batch size increase. This heavy I/O overhead contrasts sharply with the millisecond-level GPU computation time, causing severe resource underutilization (e.g., GPU utilization dropping to ~40% [51]) and rendering the inference latency unacceptable.

2.2.3 Production RDMA over VPC Systems. Production systems such as AWS EFA with Nitro [11, 23] and Alibaba Cloud eRDMA [4] already provide high-performance RDMA-like communication in cloud environments. AWS EFA relies on Nitro-based infrastructure to expose a specialized high-performance communication interface for AWS cloud instances, while Alibaba Cloud eRDMA reuses VPCs as the underlying link for cloud RDMA. These systems show that production clouds can support RDMA-like communication through specialized cloud infrastructure.

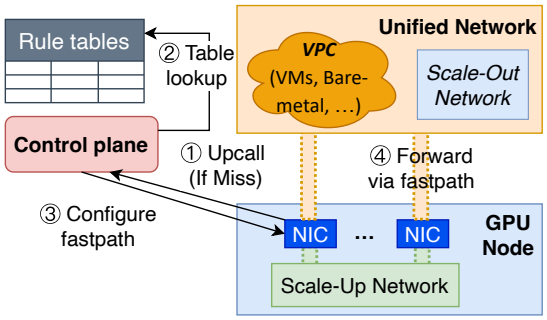

Figure 3: Architecture overview.

However, these designs are built around provider-specific infrastructure and do not directly apply to existing AI back-ends built from commodity NICs. Adopting a similar approach would require replacing back-end NICs or moving a large VPC rule database into each endpoint, both of which conflict with the cost and deployment goals of commodity-NIC AI clusters.

2.2.4 Programmable NICs. Commodity NICs are becoming increasingly programmable, but vendors expose this programmability through different hardware models. Some designs prioritize P4-programmable data planes, while NVIDIA is promoting Data Path Accelerator (DPA) [40] in its SuperNICs [37].

A representative approach for bringing VPC functions onto programmable NICs is to statically offload vSwitch-like logic onto the NIC. In such a design, a DPA-resident agent can process VPC traffic locally, including routing lookups, ACL rule matching, and tunnel encapsulation. This can provide low-latency data-plane processing, but it also changes the resource requirement of the back-end NIC.

This approach faces two resource limits. First, placing the full VPC forwarding logic on the DPA requires the NIC to store large routing and policy tables locally. At cloud scale, this can exceed on-NIC memory, and spilling state into host memory would conflict with the non-intrusiveness requirement of bare-metal deployments. Second, the DPA relies on embedded cores with a much smaller compute budget than host CPUs or DPU-class processors. Executing complex lookup, update, and slow-path logic entirely on the DPA can therefore become a throughput bottleneck as tenant scale or policy complexity grows. These limits suggest that the back-end NIC should cache only the forwarding state needed by active traffic, rather than hosting the full VPC logic locally.

2.3 Design Goals

Unifying the front-end and back-end networks is the starting point of our design. To ensure practical deployability within modern AI clusters and public clouds, the system also needs to meet the following design goals:

VPC Semantics. The back-end should support the VPC-facing semantics needed for direct access to cloud resources, including tenant isolation and policy-controlled forwarding, without moving the complete VPC policy engine into every NIC.

Non-Intrusiveness for Bare-Metal. Resource usage must stay within infrastructure boundaries and avoid host-resource intrusion, preserving bare-metal guarantees.

Minimal Impact on AI Workloads. Network functions should not become a bottleneck for throughput-sensitive AI services.

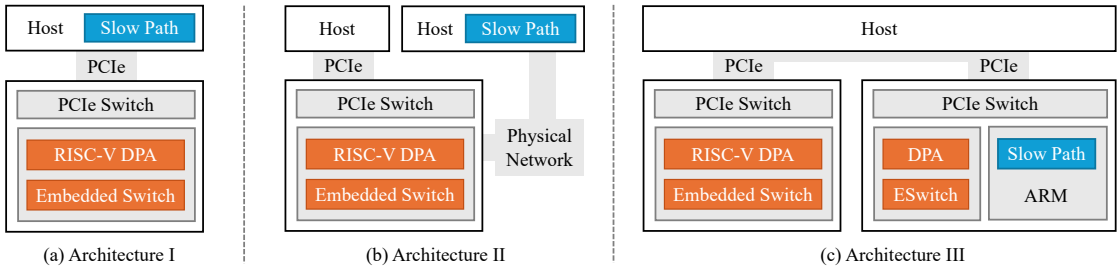

Figure 4: Deployment options.

3 System Design

Back-end NICs are optimized for high-performance data transmission, but they lack the rule-processing capabilities needed for VPC functions and direct access to VPC resources. Traditional cloud networks provide these functions through a vSwitch, often running on the host or a front-end DPU, that performs routing, ACL, and VM-server mapping lookups [50]. In a bare-metal AI back-end, however, the full vSwitch logic cannot simply be moved to an existing endpoint. Placing it on the host would consume tenant-controlled resources, while placing it on the commodity NIC would exceed the NIC's limited compute and memory budget.

The core design challenge is therefore to place VPC rule lookup outside the tenant host while keeping the commodity NIC on a lightweight forwarding fast path. One possible solution is to upgrade all back-end NICs to DPUs, allowing each DPU to store tenant-specific rule tables and perform lookups locally. This path is expensive, however, and introduces the deployment issues identified in §2.1.2.

3.1 VPC-Facing Back-end Architecture

To reduce the front-end bottleneck while preserving commodity-NIC forwarding, we propose a VPC-facing back-end architecture based on separating policy lookup from packet forwarding. The physical underlay address of the NIC is used only for back-end routing, while each GPU node attached to a VPC is assigned an independent private address in the tenant context. This separation avoids conflicts between physical and virtual addressing and allows GPUs in the AI cluster to access VPC storage and compute services through the back-end NIC.

On this basis, we design the core architecture shown in Figure 3, offloading the storage and lookup of rule tables from the NIC to the control plane and establishing a forwarding mechanism that coordinates the fast path (NIC-local flow table) and the slow path (remote controller).

Step 1. When a packet enters the NIC, it is first quickly matched locally; if there is a miss in the flow table, the packet is captured by an intermediary agent and forwarded to the remote controller.

Step 2. The controller then performs a global rule lookup based on the packet's five-tuple and the user's configured ACLs and security group rules.

Step 3. After matching, the controller generates the corresponding match-action entries and issues them to the intermediary agent, which writes them into the NIC's hardware flow table.

Step 4. Subsequent packets of the same flow can then be forwarded directly in hardware via the fast path without returning to the remote controller, significantly improving forwarding efficiency and reducing latency.

3.2 Deployment Options

We take NVIDIA NICs and DPUs as an illustrative example, using the Spectrum-X embedded switch as the fast path and the DPA as the intermediary agent. In this scenario, the physical placement of the controller becomes a key variable affecting system performance and reliability. Because the controller logic is independent of the NIC, we compare three deployment options as shown in Figure 4.

On the host CPU. Deploying on the host CPU is convenient but consumes valuable user-space resources and interferes with application processes, which conflicts with the goals of bare-metal delivery and is therefore only a theoretical alternative.

On a remote server CPU. Deployment on a remote server CPU provides good resource isolation, but control signaling must traverse the physical network, introducing potential latency concerns.

On the front-end DPU's ARM processor. Running the controller directly on the front-end DPU's ARM core can deliver low-latency interactions via a dedicated control link. However, it also creates a single failure point because a front-end DPU failure could affect all downstream NICs under its control.

3.3 Distributed Control Plane

To balance latency sensitivity and system robustness, we use a hybrid controller placement strategy with local preference and remote failover. Under normal operation, the system prefers to run the controller on the ARM processor of the front-end DPU (option c), using its dedicated control link for lower control-plane latency. Each intermediary agent also maintains connectivity to a remote controller instance (option b), which serves as a failover target when the local controller becomes unavailable. The agent uses heartbeat monitoring to track controller availability and redirects control requests to the remote controller after a local failure. This design preserves the low latency of local control in the common case while limiting the impact of controller failures.

3.4 Response to Rule Table Updates

For most AI workloads, steady-state traffic is typically dominated by a bounded active working set rather than by the full VPC policy database. Current commodity NICs already support millions of offloaded flow entries [44], which makes capacity-driven misses less likely in the common case. However, tenant changes, policy updates, or flow-table aging may refresh or revalidate many entries within a short interval, and the resulting burst of misses and upcalls

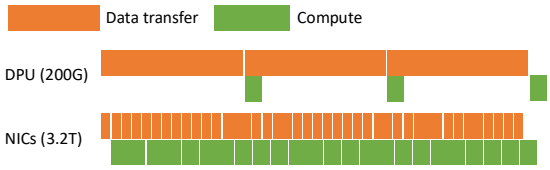

Figure 5: Comparison of data-compute overlap.

can significantly increase controller load. To avoid first-packet upcall storms caused by the traditional "delete-then-add" update, the controller precomputes and installs new rules for affected flows before removing the old ones. During updates, intermediary agents either temporarily buffer arriving packets until the new entries are in place or allow some packets to briefly follow old rules during the convergence window. This staged approach smooths bursts of upcalls from rule changes and improves forwarding continuity during updates.

In addition, to address state synchronization and cleanup of inactive flows, the system avoids costly per-entry reporting and uses batched, asynchronous notifications. The data plane periodically scans local flow activity, aggregates expired-flow information, and reports it in batches to the controller. The controller then updates its back-end flow-mapping tables based on these reports, avoiding wasted lookups and computation on torn-down flows. These optimizations bound the slow path to misses, updates, and cleanup work, while keeping common-case packets on the NIC fast path.

4 Preliminary Results

GPU Utilization. To quantify the impact of front-end bandwidth on large-scale Deep Learning Recommendation Model (DLRM) training, we build a simple analytical simulation of 64 H100 GPUs in synchronous data-parallel execution (model dimension: 1024 → 4096 → 4096 → 1024 → 1). Each server fetches a 256K-sample batch per iteration from remote storage, amounting to 512 MB per step, with a per-sample feature dimension of 1024. We assume a 3 ms compute phase, including a dense multi-layer perceptron (MLP) and all-reduce, estimated from the operation count, H100 peak floating-point operations per second (FLOPS), and an empirical model FLOPS utilization (MFU). The network is modeled with constant throughput and 80% link utilization.

We compare two data paths: (i) a disaggregated deployment with a 200 Gbps front-end DPU, and (ii) a unified front-end/back-end network using a 3.2 Tbps scale-out NIC. As shown in Figure 5, under these assumptions, loading 512 MB over a 200 Gbps link takes 25.6 ms, far exceeding the 3 ms compute window. As a result, GPUs idle for roughly 22.6 ms per iteration, reducing utilization to around 11.7%. In contrast, with 3.2 Tbps bandwidth, the data loading delay shrinks to 1.6 ms and can be almost entirely overlapped with computation. Under identical training configurations in this model, the unified network improves end-to-end throughput by 8×.

Extra Latency of First Packet. Packets that experience a match miss require controller processing before a fast-path rule can be installed, which introduces additional latency. To measure rule installation latency, we repeatedly send packets that miss the local flow table from a DPA-enabled NIC to a controller running on the host CPU. For each packet, the host controller generates a corresponding flow rule and returns it to the data plane. We measure the extra latency from the transmission of the flow-miss packet to the successful installation of the resulting rule on the DPA-enabled NIC. The experiment runs for one day, and the results are shown in Table 2. Overall, the upcall latency is on the order of tens of microseconds in this host-controller setup, indicating that first-packet processing can be kept small when the controller is placed nearby, as discussed in §5.

Table 2: Extra latency of first packets.

	P90	P95	P99	P999	P9999
Latency (μs)	50.13	54.75	62.22	71.17	266.84

5 Conclusion and Discussion

In this paper, we propose a VPC-facing back-end architecture that integrates high-performance AI clusters into VPC environments by decoupling rule lookup logic from packet forwarding. By moving policy lookup to a controller while keeping common-case forwarding on commodity NICs, the design explores a practical split between VPC-facing functionality and high-speed AI traffic. Preliminary results show that the approach can mitigate the front-end I/O bottleneck in our model and keep nearby first-packet rule installation latency in the tens of microseconds. These results support the feasibility of combining VPC-facing semantics with the performance properties of bare-metal AI fabrics, while leaving full production validation to future work.

While promising, this approach presents several challenges that warrant further research and must be addressed for deployment in production environments.

Controller Placement Strategy. The controller in our design is a software component whose main responsibility is flow-table configuration. This modular structure allows for several deployment strategies. It can be co-located with an existing controller to leverage established infrastructure and capabilities. Alternatively, it can be deployed as a standalone controller, enabling greater development agility and independent evolution. For scenarios demanding higher performance, it can also be integrated into high-performance gateways such as Sirius [13] or Sailfish [45] to achieve higher throughput. A possible direction for future work is the exploration of hardware acceleration to boost the controller's throughput while minimizing its processing latency.

Stateful Network Functions. The agent's limited computational and storage resources make it challenging to locally implement sophisticated stateful network functions (NFs) such as stateful firewalls (enforcing packet forwarding based on connection states like NEW and ESTABLISHED), flow-level rate limiting (requiring real-time bandwidth tracking), stateful packet inspection (requiring TCP sequence number validation), and connection tracking (maintaining TCP lifecycle states such as FIN/RST). Such NFs necessitate maintaining extensive per-flow state and performing complex, state-dependent logic for each packet. To address this limitation, our proposed architecture could offload the state maintenance and stateful packet processing logic to the front-end DPU or another stronger infrastructure component. This approach keeps the commodity-NIC fast path lightweight while placing demanding stateful logic on a component with more suitable compute and memory resources.

Acknowledgments

We are sincerely grateful to all anonymous reviewers for their valuable and constructive comments. This work is supported by the National Natural Science Foundation of China (Grant No. 62472404) and Alibaba Research Intern Project.

References

[1] Alibaba Cloud. 2024. Security Groups Overview. https://www.alibabacloud.com/help/en/ecs/user-guide/overview-44. Accessed: 2025-07-01.
[2] Alibaba Cloud. 2025. ECS Bare Metal Instance. https://www.alibabacloud.com/en/product/ebm?_p_lc=1. Accessed: 2025-06-24.
[3] Alibaba Cloud. 2025. Elastic Compute Service (ECS). https://www.alibabacloud.com/en/product/ecs?_p_lc=1. Accessed: 2025-06-24.
[4] Alibaba Cloud. 2025. elastic Remote Direct Memory Access network communication-eRDMA. https://www.alibabacloud.com/help/en/ecs/user-guide/elastic-rdma-erdma/. Accessed: 2026-04-28.
[5] Alibaba Cloud. 2025. Network ACL Overview. https://www.alibabacloud.com/help/en/vpc/user-guide/network-acl-overview/. Accessed: 2025-06-24.
[6] Alibaba Cloud. 2025. Object Storage Service (OSS). https://www.alibabacloud.com/en/product/object-storage-service?_p_lc=1. Accessed: 2025-06-24.
[7] Amazon Web Services. 2024. Amazon VPC Security Groups. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html. Accessed: 2025-07-01.
[8] Amazon Web Services. 2025. Amazon Elastic Compute Cloud (EC2). https://aws.amazon.com/ec2/. Accessed: 2025-06-24.
[9] Amazon Web Services. 2025. Amazon Simple Storage Service (S3). https://aws.amazon.com/s3/. Accessed: 2025-06-24.
[10] Amazon Web Services. 2025. Network ACLs - Amazon VPC. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html. Accessed: 2025-06-24.
[11] Amazon Web Services. 2026. Elastic Fabric Adapter for AI/ML and HPC workloads on Amazon EC2. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/efa.html. Accessed: 2026-04-28.
[12] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. 2025. Qwen2.5-VL Technical Report. arXiv:2502.13923 [cs.CV] https://arxiv.org/abs/2502.13923
[13] Deepak Bansal, Gerald DeGrace, Rishabh Tewari, Michal Zygmunt, James Grantham, Silvano Gai, Mario Baldi, Krishna Doddapaneni, Arun Selvarajan, Arunkumar Arumugam, Balakrishnan Raman, Avijit Gupta, Sachin Jain, Deven Jagasia, Evan Langlais, Pranjal Srivastava, Rishiraj Hazarika, Neeraj Motwani, Soumya Tiwari, Stewart Grant, Ranveer Chandra, and Srikanth Kandula. 2023. Disaggregating Stateful Network Functions. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 1469–1487. https://www.usenix.org/conference/nsdi23/presentation/bansal
[14] Aninda Chatterjee and Vivek V. 2024. Designing Data Centers for AI Clusters. https://www.juniper.net/documentation/us/en/software/nce/ai-clusters-data-center-design/ai-clusters-data-center-design.pdf. Accessed: 2025-06-21.
[15] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. 2024. RDMA over Ethernet for Distributed Training at Meta Scale. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) *(ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 57–70. doi:10.1145/3651890.3672233
[16] Google Cloud. 2024. Cloud Storage Access Control Lists (ACLs). https://cloud.google.com/storage/docs/access-control/lists. Accessed: 2025-07-01.
[17] Google Cloud. 2024. Cross-silo and cross-device federated learning on Google Cloud. https://cloud.google.com/architecture/cross-silo-cross-device-federated-learning-google-cloud. Accessed: 2025-07-02.
[18] Google Cloud. 2024. Update a Google Group to a Security Group. https://cloud.google.com/identity/docs/how-to/update-group-to-security-group. Accessed: 2025-07-01.
[19] Google Cloud. 2025. Google Cloud Bare Metal Solution. https://cloud.google.com/bare-metal. Accessed: 2025-06-24.
[20] Google Cloud. 2025. Google Cloud Compute Products. https://cloud.google.com/products/compute?hl=zh_cn. Accessed: 2025-06-24.
[21] Google Cloud. 2025. Google Cloud Storage. https://cloud.google.com/storage. Accessed: 2025-06-24.
[22] Horovod Contributors. 2025. Horovod: Distributed Deep Learning Framework. https://github.com/horovod/horovod. Accessed: 2025-06-24.
[23] Anthony Liguori. 2018. The Nitro Project–Next Generation AWS Infrastructure. Hot Chips: A Symposium on High Performance Chips. https://www.old.hotchips.org/hc31/HC31_T1_AWS_Nitro_Hot_Chips_20190818-2.pdf
[24] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruction Tuning. arXiv:2304.08485 [cs.CV] https://arxiv.org/abs/2304.08485
[25] Jiachen Liu, Fan Lai, Yinwei Dai, Aditya Akella, Harsha V. Madhyastha, and Mosharaf Chowdhury. 2023. Auxo: Efficient Federated Learning via Scalable Client Clustering. In *Proceedings of the 2023 ACM Symposium on Cloud Computing* (Santa Cruz, CA, USA) *(SoCC '23)*. Association for Computing Machinery, New York, NY, USA, 125–141. doi:10.1145/3620678.3624651
[26] Shijie Liu, Nan Zheng, Hui Kang, Xavier Simmons, Junjie Zhang, Matthias Langer, Wenjing Zhu, Minseok Lee, and Zehuan Wang. 2024. Embedding Optimization for Training Large-scale Deep Learning Recommendation Systems with EMBark. In *Proceedings of the 18th ACM Conference on Recommender Systems* (Bari, Italy) *(RecSys '24)*. Association for Computing Machinery, New York, NY, USA, 622–632. doi:10.1145/3640457.3688111
[27] Jianbiao Mei, Yukai Ma, Xuemeng Yang, Licheng Wen, Xinyu Cai, Xin Li, Daocheng Fu, Bo Zhang, Pinlong Cai, Min Dou, Botian Shi, Liang He, Yong Liu, and Yu Qiao. 2024. Continuously Learning, Adapting, and Improving: A Dual-Process Approach to Autonomous Driving. arXiv:2405.15324 [cs.RO] https://arxiv.org/abs/2405.15324
[28] Rui Miao, Lingjun Zhu, Shu Ma, Kun Qian, Shujun Zhuang, Bo Li, Shuguang Cheng, Jiaqi Gao, Yan Zhuang, Pengcheng Zhang, Rong Liu, Chao Shi, Binzhang Fu, Jiaji Zhu, Jiesheng Wu, Dennis Cai, and Hongqiang Harry Liu. 2022. From luna to solar: the evolutions of the compute-to-storage networks in Alibaba cloud. In *Proceedings of the ACM SIGCOMM 2022 Conference* (Amsterdam, Netherlands) *(SIGCOMM '22)*. Association for Computing Machinery, New York, NY, USA, 753–766. doi:10.1145/3544216.3544238
[29] Microsoft Azure. 2024. Access Control in Azure Data Lake Storage Gen2. https://learn.microsoft.com/en-us/azure/storage/blobs/data-lake-storage-access-control. Accessed: 2025-07-01.
[30] Microsoft Azure. 2024. Azure Network Security Groups Overview. https://learn.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview. Accessed: 2025-07-01.
[31] Microsoft Azure. 2025. Azure Bare Metal Infrastructure. https://learn.microsoft.com/en-us/azure/baremetal-infrastructure/concepts-baremetal-infrastructure-overview. Accessed: 2025-06-24.
[32] Microsoft Azure. 2025. Azure Blob Storage. https://azure.microsoft.com/en-us/products/storage/blobs. Accessed: 2025-06-24.
[33] Microsoft Azure. 2025. Azure Virtual Machines. https://azure.microsoft.com/en-us/products/virtual-machines. Accessed: 2025-06-24.
[34] Juniper Networks. 2025. AI Data Center Network with Juniper Apstra, NVIDIA GPUs, and WEKA Storage—Juniper Validated Design (JVD). https://www.juniper.net/documentation/us/en/software/jvd/jvd-ai-dc-apstra-nvidia-weka/jvd-ai-dc-apstra-nvidia-weka.pdf. Accessed: 2025-06-21.
[35] NVIDIA. 2020. NVIDIA MELLANOX CONNECTX-6 DX HPE ETHERNET SMARTNIC. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/documents/nvidia-connectx-6-dx-en-hpe-datasheet.pdf. Accessed: 2025-06-21.
[36] NVIDIA. 2021. NVIDIA BLUEFIELD-3 DPU PROGRAMMABLE DATA CENTER INFRASTRUCTURE ON-A-CHIP. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/documents/datasheet-nvidia-bluefield-3-dpu.pdf. Accessed: 2025-06-21.
[37] NVIDIA. 2023. What Is a SuperNIC? https://blogs.nvidia.com/blog/what-is-a-supernic/. Accessed: 2025-06-24.
[38] NVIDIA. 2024. ConnectX-7 400G Adapters. https://resources.nvidia.com/en-us-accelerated-networking-resource-library/connectx-7-datasheet. Accessed: 2025-06-21.
[39] NVIDIA. 2025. ConnectX-8 SuperNIC. https://resources.nvidia.com/en-us-accelerated-networking-resource-library/connectx-datasheet-c. Accessed: 2025-06-21.
[40] Nvidia. 2025. DPA Subsystem. https://docs.nvidia.com/doca/sdk/dpa+subsystem/index.html. Accessed: 2025-06-24.
[41] NVIDIA. 2025. NCCL: NVIDIA Collective Communications Library. https://github.com/NVIDIA/nccl. Accessed: 2025-06-24.
[42] NVIDIA. 2025. NVIDIA DOCA Software Framework. https://developer.nvidia.com/networking/doca. Accessed: 2025-07-08.
[43] NVIDIA. 2025. NVIDIA NVLink and NVLink Switch. https://www.nvidia.com/en-us/data-center/nvlink/. Accessed: 2025-06-24.
[44] NVIDIA. 2025. OVS Offload Using ASAP2 Direct. https://docs.nvidia.com/networking/display/mlnxenv495100/ovs+offload+using+asap2+direct. Accessed: 2026-03-02.
[45] Tian Pan, Nianbing Yu, Chenhao Jia, Jianwen Pi, Liang Xu, Yisong Qiao, Zhiguo Li, Kun Liu, Jie Lu, Jianyuan Lu, Enge Song, Jiao Zhang, Tao Huang, and Shunmin Zhu. 2021. Sailfish: accelerating cloud-scale multi-tenant multi-service gateways with programmable switches. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference* (Virtual Event, USA) *(SIGCOMM '21)*. Association for Computing Machinery, New York, NY, USA, 194–206. doi:10.1145/3452296.3472889
[46] AMD Pensando. 2024. AMD PENSANDO® 2 ND GENERATION ("ELBA") DATA PROCESSING UNIT. https://www.amd.com/content/dam/amd/en/documents/

pensando-technical-docs/product-briefs/pensando-elba-product-brief.pdf. Accessed: 2025-06-21.

[47] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, Chao Wang, Peng Wang, Pengcheng Zhang, Xianlong Zeng, Eddie Ruan, Zhiping Yao, Ennan Zhai, and Dennis Cai. 2024. Alibaba HPN: A Data Center Network for Large Language Model Training. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) *(ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 691–706. doi:10.1145/3651890.3672265

[48] The DPDK Community. 2025. Data Plane Development Kit Documentation. https://doc.dpdk.org. Accessed: 2025-07-08.

[49] UCX Consortium. 2025. Unified Communication X (UCX). https://openucx.org/. Accessed: 2025-06-24.

[50] Chengkun Wei, Xing Li, Ye Yang, Xiaochong Jiang, Tianyu Xu, Bowen Yang, Taotao Wu, Chao Xu, Yilong Lv, Haifeng Gao, Zhentao Zhang, Zikang Chen, Zeke Wang, Zihui Zhang, Shunmin Zhu, and Wenzhi Chen. 2023. Achelous: Enabling Programmability, Elasticity, and Reliability in Hyperscale Cloud Networks. In *Proceedings of the ACM SIGCOMM 2023 Conference* (New York, NY, USA) *(ACM SIGCOMM '23)*. Association for Computing Machinery, New York, NY, USA, 769–782. doi:10.1145/3603269.3604859

[51] Yongtong Wu, Shaoyuan Chen, Yinmin Zhong, Rilin Huang, Yixuan Tan, Wentao Zhang, Liyue Zhang, Shangyan Zhou, Yuxuan Liu, Shunfeng Zhou, Mingxing Zhang, Xin Jin, and Panpan Huang. 2026. DualPath: Breaking the Storage Bandwidth Bottleneck in Agentic LLM Inference. arXiv:2602.21548 [cs.DC] https://arxiv.org/abs/2602.21548

[52] Jiakang Yuan, Bo Zhang, Xiangchao Yan, Tao Chen, Botian Shi, Yikang Li, and Yu Qiao. 2023. AD-PT: Autonomous Driving Pre-Training with Large-scale Point Cloud Dataset. arXiv:2306.00612 [cs.CV] https://arxiv.org/abs/2306.00612

[53] Yuanxing Zhang, Langshi Chen, Siran Yang, Man Yuan, Huimin Yi, Jie Zhang, Jiamang Wang, Jianbo Dong, Yunlong Xu, Yue Song, Yong Li, Di Zhang, Wei Lin, Lin Qu, and Bo Zheng. 2022. PICASSO: Unleashing the Potential of GPU-centric Training for Wide-and-deep Recommender Systems. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, Los Alamitos, CA, USA, 3453–3466. doi:10.1109/ICDE53745.2022.00324

[54] Mark Zhao, Niket Agarwal, Aarti Basant, Buğra Gedik, Satadru Pan, Mustafa Ozdal, Rakesh Komuravelli, Jerry Pan, Tianshu Bao, Haowei Lu, Sundaram Narayanan, Jack Langman, Kevin Wilfong, Harsha Rastogi, Carole-Jean Wu, Christos Kozyrakis, and Parik Pol. 2022. Understanding Data Storage and Ingestion for Large-Scale Deep Recommendation Model Training: Industrial Product. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) *(ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 1042–1057. doi:10.1145/3470496.3533044